Ders 6 Genişletilmiş Notu: Doğrusal Regresyon, Gradyan İnişi ve Lojistik Regresyon

Ders 6 Genişletilmiş Notu: Doğrusal Regresyon, Gradyan İnişi ve Lojistik Regresyon

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sınıflandırmadan Regresyona
- 2. Regresyon Problemi: Girişten Çıkışa Bir Eşleme
- 3. Hata ve Hata Kareleri Toplamı
- 4. Gradyan İnişinin Sezgisi
- 5. Tek Özellikli ve Çok Özellikli Doğrusal Model
- 6. Öğrenme Oranı
- 7. Online Doğrusal Regresyon Algoritması
- 8. Güncelleme Kuralı Nereden Geliyor?
- 9. Batch, Stokastik ve Mini-Batch Gradyan İnişi
- 10. Lojistik Regresyon: Skor Üreten Bir Sınıflandırıcı
- 11. Skor mu, Kesin Karar mı?
- 12. Doğrusal Karar Sınırı ve Sigmoid Fonksiyonu
- 13. Olasılık Yorumu ve Eşik Değeri
- 14. Lojistik Regresyonun Öğrenilmesi

1. Sınıflandırmadan Regresyona

Hoca derse bugüne kadar gördüğümüz bütün konuların sınıflandırma (classification) üzerine olduğunu hatırlatarak başladı. Bir kişinin yüz ifadesinden mutlu mu kızgın mı olduğunu anlamak istediğimizde çıkışta sınıflar vardır: mutlu olmak bir sınıf, kızgın olmak başka bir sınıftır. Kullandığımız öğrenme modeliyle bir yüz ifadesinin hangi sınıfa ait olduğunu bulmaya çalışırız. Bir e-postanın spam olup olmadığına karar verirken de aynı şekilde iki sınıf vardır: spam olan ve olmayan e-postalar.

Fakat her problem bir sınıf kararıyla bitmez. Bir filmin puanını (rating) bilmek isteyebiliriz: 4 mü olacak, 5 mi? Ya da bir bölgedeki evin fiyatını tahmin etmek isteyebiliriz. Bu problemlerde çıkış bir sınıf değil, bir değerdir. Hoca bu tür problemlerin regresyon (regression) problemi olduğunu söyledi.

İki problem türü arasındaki fark girişte değil, çıkıştadır:

- **Sınıflandırmada** giriş verisi ayrık (discrete) da olabilir, sürekli (continuous) da. Ses tanımada giriş sürekli bir sinyaldir; sesten kişinin kim olduğunu bulmaya çalışırız. Kişi tanımada ise boy, kilo, yaş gibi özellikler kullanılabilir. Hangisi olursa olsun, çıkışta bir sınıf isteriz: bu ses kime ait, bu e-posta spam mı?
- **Regresyonda** da giriş ayrık ya da sürekli olabilir. Örneğin bir bölgedeki hava sıcaklıkları giriş olabilir. Ama çıkışta sürekli bir bilgi isteriz. Bir filmin puanı için “3’tür, 3,5’tir, 4’tür” gibi bir değer söyleriz. Artık bir sınıf söylemeyiz; çıkış sürekli bir değer olmuştur.

2. Regresyon Problemi: Girişten Çıkışa Bir Eşleme

2.1 Beşiktaş'ta ev fiyatı örneği

Hoca tahtaya bir grafik çizdi. Yatay eksende yıl bilgisi, dikey eksende Beşiktaş'taki ev fiyatları vardı. Geçmiş yıllarda ölçülmüş fiyatlar grafikteki noktalardır ve bunlar bizim eğitim örneklerimizdir. Hoca örneğin biraz yapay olduğunu da belirtti: paranın değeri düştüğü için fiyat hep artıyor gibi görünür, oysa oransal artış o kadar büyük olmayabilir.

Soru şudur: 2020 yılında Beşiktaş'ta bir evin fiyatı ne olacak? Regresyonla yapılan iş, verilen x değerlerine göre y çıkışının ne olacağını gösteren bir f fonksiyonu öğrenmektir:

$$y = f(x)$$

Bu fonksiyonu öğrendikten sonra 2020 için x değerini verip fiyatı tahmin ederiz. Doğrusal regresyonda (linear regression) bu fonksiyon doğrusaldır; yani verilere bir doğru uydurarak tahmin yaparız.

2.2 Eğitim kümesi ve eşleme

Elimizde önceden x girişleri ve bu girişler için olması gereken y çıkışları vardır. Bunlar eğitim kümemizdir (training set). N tane eğitim örneğimiz olsun:

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$$

Yapmak istediğimiz, verilen x'ler için istenen y'lere ulaşmamızı sağlayan bir eşleme (mapping) kurmaktır. Bütün (x_i, y_i) ikilileri için x_i girişinden uygun y_i çıkışına giden bu eşlemeyi arıyoruz.

2.3 Bağımsız ve bağımlı değişkenler

Hoca x'lere bağımsız değişken (independent variable), y'lere bağımlı değişken (dependent variable) dedi. Ev fiyatı örneğinde yıllar bağımsız bilgilerdir. Ev fiyatları ise yıla bağlıdır, yani bağımlı değişkendir.

Karışıklık olmasın diye hoca, modelin hesapladığı çıkışı $\hat{y}$ ("y şapka", hocanın deyişiyle "y üstü") ile gösterdi. y ise eğitim örneğinde verilen gerçek çıkıştır. Girişten çıkışa ulaşmak için bir dönüşüm kullanırız. Bu dönüşümde kullandığımız değişkenler ağırlıklardır (weights). Bir de hata terimi vardır.

3. Hata ve Hata Kareleri Toplamı

3.1 Hata nedir?

Bir örnek için hata, istenen çıkışla hesaplanan çıkış arasındaki farktır:

$$y_i - \hat{y}_i$$

Burada y_i istenen çıkış, $\hat{y}_i$ modelin hesapladığı çıkıştır. Hatanın negatif ya da pozitif olması bizim için önemli değildir; önemli olan büyüklüğüdür. Bu yüzden farkın mutlak değeri ya da karesi alınır.

Hedefimiz hatayı minimum yapmaktır. Hoca bunu öğrenmenin tanımı olarak vurguladı:

> **Hocanın vurgusu:** Öğrenme, hatayı minimum yapan uygun ağırlık değerlerini bulmaktır. Burada yapmaya çalıştığımız şey ağırlık değerlerini hatayı minimize edecek şekilde tespit etmektir.

3.2 Hata kareleri toplamı

Hata hesabı için kullanılan bağıntı, hataların karelerinin toplamıdır (sum of squared errors). N tane eğitim örneği için:

$$E = \sum_{i=1}^{N} \left(y_i - f(x_i)\right)^2$$

Doğrusal modelde $f(x_i) = v^T x_i$ olduğu için (ayrıntısı 5. bölümde):

$$E(v) = \sum_{i=1}^{N} \left(y_i - v^T x_i\right)^2$$

Hatayı minimize etmek istiyoruz. Bu ifadenin ağırlıklara göre türevini alıp sıfıra eşitlersek minimum noktaya ulaşırız.

4. Gradyan İnişinin Sezgisi

4.1 Ağırlık-hata grafiği

Hatayı minimize etmek için kullanılan yöntem gradyan inişidir (gradient descent). Hoca bunu anlatmak için bir grafik çizdi: yatay eksende ağırlık değerleri, dikey eksende hata miktarı.

Öğrenme şöyle ilerler. Başlangıçta elimizde yalnızca x'ler ve y'ler vardır. Öyle bir $\hat{y}$ bulmak istiyoruz ki ağırlık değerleri çok uygun olsun ve $y - \hat{y}$ hatası minimize olsun. Bunun için ağırlık değerlerini iteratif olarak değiştiririz:

- İşe başladığımızda yüksek bir hatayla başlarız. Örneğin ilk ağırlık değeri v_1 için hata büyüktür.
- Sonra bakılan v_2 değeri için hata biraz daha küçülür.
- Hata böyle adım adım azalır ve sonunda en düşük değere ulaşır. Bu nokta global minimumdur (global minimum).
- Bu noktadan sonra aynı yönde devam edersek hata yeniden artmaya başlar.

Gradyan inişiyle yaptığımız şey, her adımda ağırlıkları hataya göre değiştirmektir. Gradyan, ağırlıkları hangi yönde değiştireceğimizi gösterir. Ağırlıkları hata minimum olana kadar iteratif olarak değiştiririz.

4.2 Yerel minimum ve adım büyüklüğü

Bizim için önemli olan global minimuma ulaşmaktır. Hoca grafikte bazı çukurları gösterip bunların yerel minimum (local minimum) olduğunu söyledi. Yerel minimum, hatanın yakın çevresine göre en düşük olduğu ama global minimum olmayan noktadır. Bunlara takılıp kalmamamız gerekir.

Bir de tersi tehlike vardır: global minimuma gelmişken onu bir adımda atlayıp öbür tarafa geçmek. Bu yüzden ağırlıkları iteratif olarak değiştirirken dengeyi iyi yakalamak gerekir:

- **Çok küçük adımlar:** Bir önceki ağırlık 3 iken 3,05, sonra 3,1, sonra 3,15 yaparsak doğru noktaya ulaşırız ama iterasyon sayısı çok artar, öğrenme uzun sürer.
- **Çok büyük adımlar:** Uzun sürmesin diye ağırlığı büyük değiştirirsek global minimumu atlayıp başka bir noktaya geçebiliriz.

Bu dengenin nasıl kurulacağı öğrenme oranıyla ilgilidir (6. bölüm).

5. Tek Özellikli ve Çok Özellikli Doğrusal Model

5.1 Tek özellik

Hoca önce tek bir özelliğimiz olduğunu varsaydı. Örneğin bir kişinin boyuna bakıp yaşını tahmin ediyoruz (hoca bunun "enteresan bir şey" olduğunu söyleyerek güldü) ya da yıla bakarak

Beşiktaş'taki evin fiyatını tahmin ediyoruz. Tek özellik varsa tek ağırlık vardır:

$$y = v_1 x + \varepsilon$$

Burada y gerçek çıkıştır. $v_1 x$ hesaplanan çıkıştır ($\hat{y}$). ε ise hatadır. Hesaplanan çıkışa hatayı eklersek gerçek çıkışı elde ederiz.

Bu modele bir de sabit terim v_0 eklenir:

$$y = v_0 + v_1 x + \varepsilon$$

Hoca v_0'ın rolünü "doğrunun orijine oturmasını sağlamak" diye anlattı. (Bu ifadenin düzeltmesi Ders Sonrası Notlar'dadır: v_0 aslında doğrunun orijinden geçmek zorunda kalmamasını sağlar.)

Hoca tek özellik için ağırlıkların hesaplanışını da sözlü olarak verdi:

$$v_0 = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - v_1 x_i \right), \qquad v_1 = \frac{\sum_{i=1}^{N} x_i \left(y_i - v_0 \right)}{\sum_{i=1}^{N} x_i^2}$$

Bu iki denklem, hata kareleri toplamının v_0'a ve v_1'e göre türevini sıfıra eşitlemekten gelir. Örneğin v_1'e göre türev sıfıra eşitlenirse $\sum_i x_i(y_i - v_0 - v_1 x_i) = 0$ olur; buradan v_1 çekilince yukarıdaki ifade bulunur (ara adım eklendi). İki denklem birbirine bağlıdır, birlikte çözülür. Hoca bu ağırlıkları iteratif olarak hesaplamaya çalışacağımızı söyledi.

5.2 Birden fazla özellik

Birden fazla özellik varsa ne olur? Hoca film puanı örneğini verdi: filmin çevrildiği yıl, türü, oyuncuları gibi bir sürü özellik vardır. K tane özelliğimiz olsun:

$$y = v_0 + v_1 x_1 + v_2 x_2 + ... + v_K x_K + \varepsilon$$

Burada örneğin x_K filmin çevrildiği yıl, x_1 türü olabilir. Yapacağımız iş en uygun ağırlıkları belirlemektir.

5.3 Vektör gösterimi

Aradan sonra hoca modeli özetledi. x_1'den x_d'ye kadar d özellikten (number of features) oluşan bir giriş vektörümüz varsa:

$$f(x) = v_0 x_0 + v_1 x_1 + ... + v_d x_d = \sum_{j=0}^{d} v_j x_j = v^T x$$

Genel gösterimde $x_0 = 1$ kabul edilir. Yani v_0, giriş değeri her zaman 1 olan bir özelliğin ağırlığı gibi düşünülür. Hoca bunun kodlama kolaylığı için yapıldığını söyledi: Böylece sabit terim ayrı bir durum olmaktan çıkar, bütün terimler aynı toplamın içinde yazılır.

Giriş x_i aslında bir vektördür: boy, yaş, kilo gibi kaç özellik varsa o kadar bileşeni vardır. Ağırlık da bir vektördür. Hesaplanan çıkış, iki vektörün çarpımıdır: $\hat{y}_i = v^T x_i$. Hoca bunu "tek boyutlu matris gibi düşünün" diye açıkladı: $v_1 x_1 + v_2 x_2 + ...$ toplamıdır.

Doğrusal regresyonda yaptığımız iş, ağırlıkları güncelleyerek bu girişlerden istenen çıkışa en az hatayla ulaşmamızı sağlayacak y çıkışlarını hesaplamaktır.

6. Öğrenme Oranı

Ağırlıkları güncellerken kullanılan α, öğrenme oranıdır (learning rate). Öğrenme oranı ağırlıkların değişme hızını belirler: yavaş mı hızlı mı değişeceğini.

- α 1'e yaklaştıkça sistem hızlı öğrenir.
- α 0'a yaklaştıkça yavaş öğrenir.

Hoca yine hata-ağırlık grafiği üzerinden anlattı. Öğrenme oranı küçükse ağırlıklar yavaş yavaş değişir, hata da yavaş yavaş küçülür. Bu yavaş öğrenmeye yol açar ama global minimumu doğru bulmayı sağlar.

6.1 Değişken öğrenme oranı

Pratikte sıkça şu yapılır:

- Ağırlıklar ilklendirildikten sonra bir süre, örneğin ilk 100 iterasyonda, α büyük tutulur (0,8-0,9 gibi). Çünkü başta global minimumdan uzak olunduğu bilinir. Hata kocaman adımlarla değişir ve bunun bir sakıncası yoktur.
- Belli bir süre sonra öğrenme oranı küçültülür. Ağırlıklar yavaş yavaş değişmeye başlar. Bu da global minimumun kaçırılmamasını sağlar. Ağırlık birden büyük değişseydi global minimum atlanıp iterasyona devam edilebilirdi.

Hoca, pek çok çalışmada sabit bir öğrenme oranı yerine böyle değişen bir oran kullanıldığını söyledi. Bir yol, öğrenme oranını iterasyon sayısına bağlamaktır:

$$\alpha = \frac{1}{t}$$

Burada t iterasyon sayısıdır. İterasyon sayısı arttıkça α yavaş yavaş küçülür: $t = 1$ için 1, $t = 2$ için 0,5, $t = 10$ için 0,1, $t = 100$ için 0,01 olur.

Öğrenme oranı sabit de seçilebilir; hoca sabit seçilecekse 0 ile 1 arasında bir değer alınabileceğini söyledi.

> **Hocanın vurgusu:** Öğrenme oranı ağırlıkların değişme hızını belirler. Çok yavaş hareket edersek iterasyon çok fazla olur; çok hızlı hareket edersek global minimumu atlayabiliriz. Dengeyi iyi yakalamak gerekir.

7. Online Doğrusal Regresyon Algoritması

Hoca en uygun ağırlıkları bulmak için online doğrusal regresyon yöntemini anlattı. Algoritma şu adımlardan oluşur:

```
1. Ağırlıkları ilklendir:
   d özellik varsa her özellik için bir ağırlık, ayrıca v0.
   Başlangıç değerleri 0 ile 1 arasında küçük rastgele sayılar.
2. Tekrarla:
   2.1 Eğitim örneklerinden birini seç: (x_i, y_i)
   2.2 Çıkışı hesapla: f(x_i) = v^T x_i
   2.3 Ağırlıkları güncelle:
       v(t+1) = v(t) + alpha * (y_i - f(x_i)) * x_i
3. Hata belirlenen bir epsilon eşiğinin altına düşene kadar
   (ya da belirlenen iterasyon sayısına ulaşana kadar) devam et.
```

Güncelleme kuralının matematiksel yazımı:

$$v^{(t+1)} = v^{(t)} + \alpha \left(y_i - f(x_i)\right) x_i$$

Burada $v^{(t)}$, t anındaki ağırlık vektörüdür. Hoca formülü yazarken bir öğrenci bir parantezin eksik olduğunu fark etti; hoca teşekkür edip düzeltti.

7.1 Güncelleme kuralının yorumu

Hoca güncelleme kuralının ağırlıkları nasıl değiştirdiğini işaretlere bakarak açıkladı:

- $y_i - \hat{y}_i > 0$ ise, yani $y_i > \hat{y}_i$ ise (model istenenden küçük bir değer hesaplamışsa) parantezin içi pozitif çıkar. Bir önceki ağırlığa pozitif bir değer eklenir; $v^{(t+1)}$ artar.
- $y_i - \hat{y}_i < 0$ ise (model istenenden büyük bir değer hesaplamışsa) parantezin içi negatif çıkar; $v^{(t+1)}$ azalır.

"Ağırlıkların değişimi" dediğimiz şey budur: istenen ve hesaplanan çıkışın farkı pozitifse ağırlık artırılır, negatifse azaltılır. İşlem iteratif olarak tekrarlanır.

> **Ek bilgi (derste anlatılmadı):** Tek bir güncellemenin etkisini küçük bir örnekle görelim. $x_i = (1, 2)$ ($x_0 = 1$ dahil), $y_i = 3$, ağırlıklar $v = (0, 0)$ ve $\alpha = 0{,}1$ olsun. Hesaplanan çıkış 0'dır, hata $3 - 0 = 3$'tür. Yeni ağırlıklar $v = (0 + 0{,}1 \cdot 3 \cdot 1,\ 0 + 0{,}1 \cdot 3 \cdot 2) = (0{,}3;\ 0{,}6)$ olur. Yeni çıkış $0{,}3 + 0{,}6 \cdot 2 = 1{,}5$'tir: hata 3'ten 1,5'e inmiştir.

7.2 Durma koşulu

Ne zamana kadar devam edilir? Hoca genellikle yapılanın, hata belli bir ε değerinin altına düşene kadar devam etmek olduğunu söyledi. Buna yakınsama (convergence) denir. Bazı çalışmalarda ise geçen hafta konuşulduğu gibi sabit bir adım sayısı belirlenir (örneğin "10.000 adım yap").

8. Güncelleme Kuralı Nereden Geliyor?

Hoca soru olup olmadığını sorduğunda bir öğrenci şunu sordu:

Soru: Daha önce hatanın türevini alıp sıfıra eşitlemiştik, minimum noktanın türevin sıfır olduğu yer olduğunu söylemiştik. Ama bu algoritmada türevi kullanmadık sanırım?

Cevap: Hoca öğrencinin haklı olduğunu, bunu uzun olacağı için yazmadığını söyledi ve tamamladı. Bir örnek için hata şöyle yazılır:

$$E_i = \frac{1}{2} \left(y_i - v^T x_i \right)^2$$

Bunun ağırlıklara göre türevi alınır. Zincir kuralından dolayı parantezin içinin türevi, yani x_i, dışarı çıkar:

$$\frac{\partial E_i}{\partial v} = - \left(y_i - v^T x_i \right) x_i$$

Gradyan inişinde ağırlık, gradyanın tersi yönünde, α kadar hareket ettirilir:

$$v^{(t+1)} = v^{(t)} - \alpha \frac{\partial E_i}{\partial v} = v^{(t)} + \alpha \left(y_i - v^T x_i \right) x_i$$

Güncelleme kuralındaki x_i çarpanı bu türevden gelir. $\frac{1}{2}$ çarpanı da türevde gelen 2'yi sadeleştirmek için konur (ara adım eklendi). Hoca bunu özellikle ayrıntılı anlatmadığını, çünkü kafa karıştırdığını, bu yüzden yalnızca "türevi sıfıra eşitlerseniz hatayı minimize edersiniz" deyip geçtiğini söyledi. Öğrenci "şimdi gördüm" dedi; hoca da soru için teşekkür etti.

> **Hocanın vurgusu:** Soru sormanız önemli. Sizin sorduğunuz şeyi arkadaşlarınız da anlamamış olabilir, hatta anlamadıklarını fark etmemiş olabilirler.

9. Batch, Stokastik ve Mini-Batch Gradyan İnişi

Hoca gradyan inişini özetlerken yine ağırlık-hata grafiği çizdi: gradyan eğriye diktir, yola bir noktadan başlanır ve hata yavaş yavaş küçültülür. Gradyan inişini uygularken üç farklı yol vardır.

9.1 Batch gradient descent

Eğitim örneklerinin tamamı alınır. Bütün eğitim örnekleri için hata tek tek hesaplanır. Sonra hataların ortalaması alınarak ağırlıklar güncellenir (update).

Burada epoch kavramı geçer. Epoch bir turdur: 1.000 örneğimiz varsa 1.000 örneğin her biri için işlem yapıldığında bir tur tamamlanmış olur. Batch yönteminde ağırlıklar her epoch bittikten sonra bir kez güncellenir. Epoch'lar arttıkça ortalama hata düzgün bir şekilde düşer.

Hoca bunun dezavantajını sordu. Bir öğrenci "çok fazla zaman gerekiyor" dedi. Hoca bunu onaylayıp açtı:

- Örnek sayısı çok büyükse (binlerce, on binlerce) tek bir güncelleme için bütün örneklerin sisteme gösterilmesi gerekir. 10.000 örneği gösterir, bir kez güncellersiniz; 10.000 örneği bir daha gösterir, bir kez daha güncellersiniz.
- Ortalama alındığı için ağırlıklar küçük adımlarla değişir; bir adımda 3,1, sonraki adımda 3,2 olur gibi.

Bu yüzden batch yöntemi yavaş çalışır. Yine de kullanılır: Bir milyon örneğiniz yoksa batch kullanılabilir.

Hoca bunun genel bir yaklaşım olduğunu, doğrusal regresyona özgü olmadığını hatırlattı. Derin öğrenmede yapay sinir ağları da ağırlıkları güncellemek için gradyan inişi kullanır. Doğrusal regresyonda genelde o kadar çok örnekle çalışılmaz, ama bir milyon örnekle bir sinir ağı eğittiğinizi düşünün: Ağırlıkları batch yöntemiyle güncellemek büyük sorun olur.

9.2 Stokastik gradient descent

Her seferinde bir örnek gösterilir ve ağırlıklar hemen güncellenir. Bir örnek gösterilir, güncellenir; bir sonraki örnek gösterilir, yine güncellenir.

Sıkıntısı şudur: Her seferinde tek bir örneğe bakıldığı için sistemin genel durumu görülmez. Bu, genellemeye gitmeye engel olur. Bir örnekte ağırlık artar, ikincisinde düşer, üçüncüsünde düşer, dördüncüsünde artar. Hata dalgalana dalgalana ilerler. Global minimuma çok yaklaşıp bir türlü tam ulaşamayabilir. Batch yönteminde ise sistemin genel durumu görülerek güncelleme yapılabiliyordu.

Dersin sonlarında bir öğrenci "online" kelimesinin neyi kastettiğini sordu.

Soru: İlk olarak "online doğrusal regresyon" dedik. Online derken neyi kastettik, doğrusal regresyonun başka çeşitleri var mı?

Cevap: Online denilen şey stokastik gradyan inişidir: batch değil, örnekleri tek tek gösterip her örnekten sonra güncelleyen yöntem. Hoca doğrusal olmayan başka çeşitlerin de olduğunu, eskiden anlattığını ama çok kullanılmadığını söyledi.

Yani 7. bölümde yazılan algoritma aslında stokastik gradyan inişidir.

9.3 Mini-batch gradient descent

İki yöntemin yerine önerilen, mini-batch gradyan inişidir. Bir K değeri seçilir. Diyelim $K = 100$ ve bir milyon örneğimiz var. Her 100 örnekte bir, ortalama hatayla ağırlıklar güncellenir: 100 örnek için hata tek tek hesaplanır, ortalamaları alınır, ağırlıklar güncellenir, sonra bir sonraki 100 örneğe geçilir.

Böylece:

- Batch yönteminin yavaşlığından, yani bütün örnekleri gösterdikten sonra ancak bir kez güncelleme yapmaktan kurtulunur.
- Stokastik yöntemin yarattığı gereksiz dalgalanmadan kurtulunur.
- Global minimuma daha hızlı ulaşılır.

Mini-batch'te de dalgalanma olur; hata yine dalgalanarak azalır. Ama stokastik ve batch yöntemlerinden daha verimlidir.

> **Hocanın vurgusu:** Tavsiye edilen yöntem mini-batch gradyan inişidir.

9.4 Mini-batch grupları nasıl seçilir?

Bir öğrenci mini-batch'in nasıl işlediğini netleştirmek istedi.

Soru: Mini-batch'te aldığımız 100 örneği her hata hesabından sonra değiştiriyoruz, değil mi? Örneğin 1.000 örneğimiz varsa 100'lük gruplar halinde mi hata hesaplıyoruz? Yoksa tek bir grup alıp sürekli onun üzerinde mi hesaplıyoruz?

Cevap: Hoca 100'lük gruplar yapıldığını doğruladı. İlk 100 örnekle ortalama hata hesaplanır, ağırlıklar güncellenir. Sonra ikinci 100'e, üçüncü 100'e geçilir. Hep aynı grup kullanılmaz. Bütün gruplar bitince başa dönülür; hâlâ yakınsama olmadıysa, yani hata minimize edilmediyse, baştan devam edilir.

Hoca üç yöntemi bu soru üzerinden bir kez daha karşılaştırdı:

- **Stokastik:** Örneği sisteme gösterirsin, hataya göre ağırlıkları hemen güncellersin.
- **Batch:** Ağırlıkları her örnekte güncellemezsin; hatayı hesaplar, bütün tur bittiğinde güncellersin. 10.000 örneği gösterir, güncellersin, sonra aynı 10.000 örneği yeniden gösterirsin.
- **Mini-batch:** 100'ü alırsın, güncellersin, bir sonraki 100'e geçersin. Hepsi bitince başa dönersin.

10. Lojistik Regresyon: Skor Üreten Bir Sınıflandırıcı

Hoca öğrenmede en çok kullanılan modellerden birinin lojistik regresyon (logistic regression) olduğunu söyledi. İki nedenle tercih edilir: Birincisi çok basittir. İkincisi, adında "regresyon" geçse de aslında bir sınıflandırıcıdır.

Doğrusal regresyon gibi sürekli bir çıkış vermez. "Beşiktaş'taki ev 2020'de 3 milyon TL olacak" ya da "bu filmin puanı 4,5" gibi bir değer üretmez; bir sınıf verir. Fakat çıkış değeri 0 ile 1 arasında bir değer olduğu için (0,8 ya da 0,3 gibi) özellikle çıkışta skor görmek isteyenler bu yöntemi çok tercih eder.

Hoca iki uygulama örneği verdi:

- **Sahtekârlık tespiti (fraud detection):** Bir banka "bu işlem fraud'dur ya da değildir" gibi kesin bir karar istemiyor. "Bu işlemin fraud değeri 0,4'tür" ya da "0,9'dur" gibi bir skor istiyor ve kararı bu skora bakarak kendisi vermek istiyor. Hoca bankaların daha önce de fraud kararını elle, ellerindeki kurallara göre bir skor hesaplayarak verdiklerini; modele geçtiklerinde de aynı alışkanlığı sürdürdüklerini anlattı.
- **İşe alım:** Önceki yıllardaki işe alımların tecrübelerine göre, yeni başvuran bir kişinin işe alınmasının uygun olup olmadığına dair bir skor üretiliyor. Hoca, TÜBİTAK projelerinde yaptığı hakemliklerde en az 4-5 projede insan kaynaklarının işe alımda lojistik regresyon kullandığını gördüğünü söyledi. Önceki işe alımlardaki kişilerin profilleri giriş olarak kullanılıyor, yeni başvuranın profiline göre uygunluk skoru belirleniyor. Bunun son iki yılda bir trend olduğunu da ekledi.

11. Skor mu, Kesin Karar mı?

Hoca, hakemliklerde proje sahiplerine de sorduğu bir soruyu sınıfa sordu: Bu skor çıkışı uygun mudur? "İşe al" ya da "alma" gibi kesin bir karar mı daha iyi olurdu, yoksa skor mu? Doğru ya da yanlış bir cevap olmadığını, fikirlerini duymak istediğini söyledi.

Öğrencilerden gelen görüşler şunlardı:

- **Bir öğrenci:** "Al ya da alma" çok kesin olur ve hata payı daha yüksek gibi görünür; skor daha iyi.
- **İkinci öğrenci:** Skor daha iyidir, çünkü sonunda sıralama yapılır ve en yüksek skorlulara öncelik verilir. Kesin kararda 10 kişiye "al", 10 kişiye "alma" denir; skorda ise "10 üzerinden 8'leri" alırsın, daha çok kategori oluşur.
- **Fraud için (ikinci öğrenci):** Yine skoru tercih ederim ama skor olunca işin içine insan kararı girer. Örneğin standart olarak 10 üzerinden 7'nin üstü gerçekten fraud çıkıyorsa, 7'yi sınır belirleyip altını ve üstünü kendim ayırırım.

- **Üçüncü öğrenci:** Yapılan işin önemine göre değişir. İşe almada öncelik önemlidir, bu yüzden skor önemlidir. Fraud ise kesin bir durumdur; "biri daha az fraud, biri daha çok fraud" diye değerlendirme yapamayız. Bu yüzden fraud için kesin karar daha mantıklıdır.

Hoca iki görüşün de doğru olduğunu, ama üçüncü öğrencinin kendi düşündüğünü ifade ettiğini söyledi ve görüşünü açtı:

- **Sıralama gerekiyorsa skor faydalıdır.** 50 kişi başvurmuş ve 10 kişi işe alınacaksa, adayları sıralamak gerekir; yüksek skorlular seçilir.
- **İkili bir kararda skor tehlikelidir.** Tek bir kişi işe alınacağını ya da fraud kararını düşünün. Model 0,7 verdi: Alalım mı, almayalım mı? Burada aslında ikili (binary) bir karar veriyoruz. "Skoru göreyim, kararı ben vereyim" dediğimizde işin içine insan tecrübesi girer. Biri daha katı düşünüp "0,7 ise almıyorum" diyebilir. Başka biri "0,5'i aşmış, alalım" diyebilir. Farklı kişiler aynı skoru farklı yorumlar.

  Hocanın vurgusu: Skoru kullanırken dikkat etmeniz lazım. İşin içine insan muhakemesi, farklı kişilerin farklı değerlendirmeleri girecekse skor doğru değildir. İnsan faktörünü dikkatli kullanmazsanız makine öğrenmesinin size sağladığı genellemeye gitme işini bozabilirsiniz.

11.1 Eşik değerini veriyle belirlemek

Hoca, skoru kişilerin inisiyatifine bırakmak yerine eşik değerini (threshold) verilere bakarak belirlemeyi önerdi: Eşiği ne alırsak en iyi sonuçları buluruz?

Örneğin işe alımda: Skoru 0,6 olan birini işe aldım, bir yıl sonra memnun olmadım. Skoru 0,7 olanı aldım, yine memnun olmadım. Ama 100 kişiyi işe aldığımda 0,8'i geçenlerin hepsinden memnun kaldım. O zaman eşik 0,8 olmalıdır. Böylece karar yine insan muhakemesinden kurtarılmış olur.

"Memnun oldum, olmadım" demek de bir insan muhakemesi değil mi? Hoca bunun da doğru olduğunu kabul etti; ama burada etiketleme yapıldığını söyledi. Etiketlemenin en büyük sıkıntısı da etiketlenen verilerde insanlar arası farklılıklardır. Hocanın örneği duygu analiziydi (sentiment analysis): İş yerlerinde çok kullanılan bir uygulama, müşteri yorumlarını olumlu ve olumsuz diye sınıflandırmaktır. Bu yorumları insanlar etiketler ve herkesin kendine göre bir eşiği vardır; birinin olumlu dediğine başkası nötr diyebilir.

11.2 Skoru kim veriyor?

Bir öğrenci şunu sordu:

Soru: İşe alımda bir kişiden büyük beklentiyle alırsın; kişiden kendini 10 üzerinden puanlamasını istersin, 10 der ama gösterdiği performans 5 olur. Bu geri dönüşte negatif besleme yaratmaz mı? Çok yüksek skorlular kötü etki yaratmış olmaz mı?

Cevap: Hoca skoru kişinin kendisinin vermediğine dikkat çekti. Skoru model verir; etiketi de işe alan kurum verir. Önerilen model şöyledir: Daha önceki işe alımlardaki memnuniyete göre profiller etiketlenir. "Şu profildeki kişiyi aldık, memnun kaldık", "şunu aldık, memnun kalmadık", "şunu aldık, bir yıl sonra ayrıldı, bu da bir memnuniyetsizlik" gibi. Giriş bilgileri ise başvurunun özellikleridir: mezun olunan okul, not ortalaması, önceki iş tecrübeleri, konuyla ilgili tecrübe. Bunlara puanlar verilir ve sisteme giriş olarak verilir. Hoca bu puanların da çok objektif olmayabileceğini ekledi.

Hoca böyle çalışmaları beğendiğini söyledi ve çalışan öğrencilere bir tavsiyede bulundu:

> **Hocanın vurgusu:** Çalıştığınız şirkette ne kadar büyük veri varsa o kadar güzel çalışmalar çıkarılabilir. Dataya bakın; büyük veriden çok güzel tezler çıkar.

12. Doğrusal Karar Sınırı ve Sigmoid Fonksiyonu

12.1 Doğrusal karar sınırı

Lojistik regresyon doğrusal bir karar sınırı (linear decision boundary) kullanır. İki sınıfımız olsun. Karar sınırı, $v^T x = 0$ olan yerdir. Sınırın bir tarafında $v^T x > 0$, öbür tarafında $v^T x < 0$

olur.

12.2 Modelin yapısı

Hoca modeli bir şema olarak çizdi. Girişler $x_1, \dots, x_d$ ve sabit giriş $x_0 = 1$, ağırlıklar $v_0, v_1, \dots, v_d$ ile çarpılıp toplanır ve bir z değeri elde edilir:

$$z = \sum_{i=0}^{d} v_i x_i = v^T x$$

Doğrusal regresyonda çıkış doğrudan bu toplamdı. Lojistik regresyonda ise z, bir fonksiyondan geçirilir. Bu fonksiyon sigmoid fonksiyonudur, lojistik fonksiyon (logistic function) olarak da adlandırılır:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Yani modelin çıkışı:

$$f(x) = \frac{1}{1 + e^{-\sum_{i=0}^{d} v_i x_i}}$$

12.3 Sigmoid'in özellikleri

Fonksiyona sigmoid denmesinin sebebi S harfine benzer bir şekli olmasıdır. Hoca bunu basamak (step) fonksiyonuyla karşılaştırdı: Basamak fonksiyonu sıfırlardan birlere "çat diye", keskin bir şekilde geçer. Sigmoid ise yumuşak (smooth) bir geçiş sağlar; 0'dan 1'e yavaş geçilir. Böylece çıkış 0 ile 1 arasında değişen değerler alır. Daha önce "skor" dediğimiz değer budur.

> **Ek bilgi (derste anlatılmadı):** Sigmoid'in birkaç değeri şekli anlamaya yardım eder: $g(0) = 0{,}5$, $g(2) \approx 0{,}88$, $g(-2) \approx 0{,}12$. z büyüdükçe çıkış 1'e, küçüldükçe 0'a yaklaşır. Karar sınırı $v^T x = 0$ olan noktalarda çıkış tam 0,5'tir; eşik 0,5 seçildiğinde sınıflandırmanın doğrusal karar sınırına göre yapılması bu yüzdendir.

13. Olasılık Yorumu ve Eşik Değeri

13.1 Olasılık olarak çıkış

Lojistik regresyonda verilen x girişleri için 0 ile 1 arasında değişen çıkış değerleri öğrenilir. Çıkış, verilen x ve v için y'nin 1 olma olasılığı olarak yorumlanır:

$$P(y = 1 \mid x, v) = g(v^T x) = \frac{1}{1 + e^{-v^T x}}$$

y'nin 0 olma olasılığı ise:

$$P(y = 0 \mid x, v) = 1 - P(y = 1 \mid x, v)$$

Yani model size bir olasılık üretir. Örneğin "bu kişiyi 0,8 olasılıkla işe alırsan iyi olur" gibi.

Hoca bunun, şimdiye kadar gördüğümüz sınıflandırıcıların bir kısmından farkını vurguladı. Naive Bayes, k en yakın komşu ve karar ağacı kesin kararlar veriyordu: "Bu örnek A sınıfına aittir, bu B sınıfına aittir." Lojistik regresyon ise 0 ile 1 arasında bir değer üretir. Zaten ikili sınıflandırma (binary classification) için kullanılır. Örneğin fraud/non-fraud değerlendirmesinde çıkış bir tarafa yakınsa bir sınıf, öbür tarafa yakınsa diğer sınıf denebilir.

13.2 Eşik değeri

Bir eşik değeri θ belirlenir:

- $P(y = 1 \mid x, v) > \theta$ ise çıkış 1 denir.
- Değilse çıkış 0 denir.

Böylece lojistik regresyon ikili çıkış (binary output) veren bir sınıflandırıcı olarak kullanılır. Mantıken $\theta = 0{,}5$ alınabilir: 0,5'ten büyükler bir sınıfa, küçükler öbür sınıfa aittir. Ya da 11. bölümde konuşulduğu gibi eşik verilere bakarak belirlenir. Bir seçenek de çıkışı skor olarak bırakmak ve kararı kullanıcıya vermektir.

Hoca bunu bir kez daha şema üzerinden toparladı: Örneklerin olduğu düzlemde $g(v^T x)$ değerine göre bir taraf bir sınıf, öbür taraf diğer sınıftır. Model verilen x değerleri için y'nin 1 olma olasılığını verir; geri kalanı y'nin 0 olma olasılığıdır. Bu olasılık eşikten büyükse örnek 1 sınıfına, küçükse 0 sınıfına aittir.

14. Lojistik Regresyonun Öğrenilmesi

Hoca lojistik regresyonun öğrenilmesinin doğrusal regresyondan çok farklı olmadığını söyledi. Mantık aynıdır:

```
1. Ağırlıkları başlangıçta küçük değerlerle ilklendir.
2. Tekrarla:
   2.1 Bir eğitim örneği seç: (x_i, y_i)
   2.2 Çıkışı hesapla:
       f(x_i) = 1 / (1 + exp(-(v0*x0 + v1*x1 + ... + vd*xd)))
   2.3 Ağırlıkları güncelle:
       v(t+1) = v(t) + alpha * (y_i - f(x_i)) * x_i
3. Hata yeterince küçülene kadar devam et.
```

Tek fark, çıkışın artık doğrudan $v^T x$ değil, sigmoid fonksiyonundan geçirilmiş değer olmasıdır:

$$f(x_i) = \frac{1}{1 + e^{-v^T x_i}}, \qquad v^{(t+1)} = v^{(t)} + \alpha \left(y_i - f(x_i)\right) x_i$$

Derste yazılan sürüm online (stokastik) sürümdür; her örnekten sonra ağırlıklar güncellenir. Hoca bunun zorunlu olmadığını söyledi: İstenirse her gelen örnekte güncelleme yapılmaz, hata hesaplanır ve bütün örnekler gösterildikten sonra güncellenir (batch), ya da mini-batch'teki gibi gruplarla çalışılır.

14.1 Perceptron'a bağlantı

Hoca bu güncelleme mantığının hep aynı olduğunu ve ileride de karşımıza çıkacağını söyledi. Yapay sinir ağlarına (artificial neural networks) geçtiğimizde perceptron'un mantığı da böyledir; ağırlıkların güncellenmesi benzer biçimde olacaktır. Çok katmanlı perceptron'da (multilayer perceptron) da ağırlıklar başlangıç değerleriyle ilklendirilir, bu ağırlıklarla çıkış hesaplanır ve ağırlıklar güncellenir.

> **Hocanın vurgusu:** Aynı ağırlık güncelleme mantığını perceptron ve çok katmanlı perceptron konularında da göreceğiz.

Dersin Toparlaması

Bu derste sınıflandırmadan regresyona geçtik. Regresyonda çıkış bir sınıf değil, sürekli bir değerdir: bir evin fiyatı ya da bir filmin puanı. Doğrusal regresyon, girişlerle çıkış arasında $\widehat{y} = v^T x$ biçiminde doğrusal bir eşleme kurar ($x_0 = 1$ kabulüyle sabit terim de bu yazıma dahildir). Öğrenme, hata kareleri toplamını minimum yapan ağırlıkları bulmaktır. Bunun için gradyan inişi kullanılır: Ağırlıklar, hatanın türevinden gelen $v^{(t+1)} = v^{(t)} + \alpha(y_i - f(x_i))x_i$ kuralıyla iteratif olarak güncellenir. İstenen çıkış hesaplanandan büyükse ağırlık artar, küçükse azalır. Öğrenme oranı α adımın büyüklüğünü belirler; başta büyük, sonra küçülen bir oran hem hız hem de global minimumu kaçırmama açısından iyi bir dengedir.

Gradyan inişinin üç biçimini karşılaştırdık. Batch yöntemi bütün örnekleri gösterip bir kez günceller; düzgündür ama büyük veride yavaştır. Stokastik (online) yöntem her örnekte günceller; hızlıdır ama dalgalıdır. Mini-batch yöntemi K örneklik gruplarla güncelleyerek ikisinin sıkıntısını azaltır ve tavsiye edilen yöntemdir.

Son olarak lojistik regresyonu gördük. Adında regresyon geçse de bir sınıflandırıcıdır: $v^T x$ toplamını sigmoid fonksiyonundan geçirerek 0 ile 1 arasında, $y = 1$ olma olasılığı olarak yorumlanan bir değer üretir ve doğrusal bir karar sınırı kullanır. Bu değer bir eşikle ikili karara çevrilebilir ya da skor olarak bırakılabilir. Sınıf tartışmasında skorun sıralama gereken durumlarda faydalı olduğu, ama ikili kararlarda insan yorumunu işin içine sokup genellemeyi bozabileceği konuşuldu; eşiğin geçmiş sonuçlara bakılarak belirlenmesi önerildi. Lojistik regresyonun öğrenilmesi de doğrusal regresyonla aynı güncelleme mantığına dayanır. Hoca bir sonraki derste bu mantığın devamı olan yapay sinir ağlarına ve perceptron'a geçeceğini söyledi.

Ders Sonrası Notlar

Ders sonrası not: Hoca v_0'ın rolünü "doğrunun orijine oturmasını sağlamak" diye anlattı. Doğrusu tam tersidir. v_0 (sabit terim, intercept, bias) olmadan $y = v_1 x$ doğrusu her zaman orijinden geçer. v_0 eklenince doğru dikey eksende v_0 noktasından geçer; yani orijinden geçmek zorunda kalmaz.

Ders sonrası not: Hoca hatanın türevini sıfıra eşitlemeyi ve gradyan inişini aynı cümlede, sanki aynı yöntemmiş gibi anlattı ("hatayı minimize etmek için kullanılan yöntem gradient descent"). İkisi farklı yollardır. Türevi sıfıra eşitleyip denklemleri çözmek kapalı biçimde (analitik) çözümdür; tek özellik için 5.1'deki denklemler bu yoldan gelir. Gradyan inişi ise aynı minimuma türevin gösterdiği yönde küçük adımlarla, iteratif olarak yaklaşır. Online algoritmadaki güncelleme kuralı türevden gelir ama türevi sıfıra eşitlemez; türevin tersi yönünde adım atar.

Ders sonrası not: Hocanın ağırlık-hata grafiğinde çizdiği yerel minimumlar doğrusal regresyon için geçerli değildir. Kare hata fonksiyonu doğrusal regresyonda dışbükeydir (convex); tek bir global minimumu vardır ve yerel minimuma takılma sorunu yoktur. Yerel minimum sorunu, ileride görülecek çok katmanlı yapay sinir ağları gibi doğrusal olmayan modellerde ortaya çıkar. Hocanın uyarısı genel gradyan inişi için doğrudur.

Ders sonrası not: Hoca boy, kilo, yaş gibi özellikleri "ayrık (discrete) veri" diye örnekledi. Bunlar aslında sürekli (continuous) sayısal değişkenlerdir. Ayrık değişkene örnek olarak film türü, şehir ya da kişi sayısı verilebilir.

Ders sonrası not: Hoca x'lerin bağımsız değişken olmasını "birbirinden bağımsız" diye açıkladı. Regresyondaki "bağımsız değişken" ifadesi, girişlerin birbirinden istatistiksel olarak bağımsız olduğu anlamına gelmez; çıkışı açıklayan (açıklayıcı) değişkenler demektir. Girişler kendi aralarında ilişkili olabilir.

Ders sonrası not: Hoca öğrenme oranının 0 ile 1 arasında seçilebileceğini söyledi. α teorik olarak 1'den büyük de seçilebilir; ama pratikte genelde küçük değerler (0,001-0,1 gibi) kullanılır. Hocanın verdiği 0,8-0,9 başlangıç değerleri, verinin ölçeği büyükse ağırlıkların ıraksamasına yol açabilir. Uygun değer probleme ve verinin ölçeğine bağlıdır; özellikleri normalize etmek (4. dersteki normalizasyon) bu yüzden de önemlidir.

Ders sonrası not: Derste lojistik regresyon için doğrusal regresyondaki güncelleme kuralı aynen kullanıldı, ama hangi hata fonksiyonundan geldiği söylenmedi. Lojistik regresyonda genelde kare hata yerine çapraz entropi (log-loss) kullanılır: $E = -\sum_i [y_i \log f(x_i) + (1 - y_i) \log(1 - f(x_i))]$. Bu hatanın ağırlıklara göre gradyanı $-(y_i - f(x_i))x_i$ olur ve derste yazılan $v^{(t+1)} = v^{(t)} + \alpha(y_i - f(x_i))x_i$ kuralını verir. (Bu eşitlik sayısal türevle kontrol edildi.)

Ders sonrası not: Hoca lojistik regresyonun olasılık ürettiğini söyleyip hemen ardından "olasılıkla değil de, skor" diye düzeltti. İkisi de yaygın kullanılır. Çıkış olasılık

olarak yorumlanabilir, ama bu olasılığın gerçek sıklıklarla uyumlu olması (kalibrasyon) her zaman garanti değildir.

Ders sonrası not: Altyazıdaki bozulmalar: “Fırat”, “fraat”, “fraut”, “frağıt” kelimeleri “fraud” (sahtekârlık) anlamındadır, kişi adı değildir. “İPOK” epoch, “online diner regresyon” online linear regression, “Nayibayez” Naive Bayes, “mini-beç” ve “mini 5” mini-batch, “5 gradient” batch gradient, “gradient percent” gradient descent, “sıkı poit” sigmoid, “trash”/“tıraş yoltuk” threshold, “iPhone” ise α’dır. “Kostumuz” cost (maliyet) demektir. Mini-batch örneğindeki “11 tane örneğimiz var” ifadesi, hocanın hemen sonra “1 milyon tane örneğimiz vardı” demesinden anlaşıldığı gibi “1 milyon” olmalıdır.

Ders sonrası not: Hocanın hakemlik yaptığı programlar altyazıda “TÜBİTAK-TEDEF” diye geçiyor. Büyük olasılıkla TÜBİTAK TEYDEB (Teknoloji ve Yenilik Destek Programları Başkanlığı, sanayi Ar-Ge projeleri) kastediliyor.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.